GUÍA PRÁCTICA DE ALTO RENDIMIENTO

Manual de Uso de Antigravity

Cómo comunicarte, qué comandos usar y cómo aprovechar subagentes, automatizaciones y herramientas especializadas, con ejemplos listos para copiar y adaptar.

Enfoque: Productividad & Flujos Reales **Nivel:** Básico a Avanzado **Edición:** Noviembre 2026

1 La regla de oro: cómo pedir las cosas

No necesitas dictar cada paso técnico, pero la calidad del resultado depende directamente de la claridad con la que defines el encargo. Estructura tus peticiones con tres elementos:

1. Objetivo real

Qué quieres lograr y para qué sirve. Permite proponer mejores soluciones que una orden aislada.

2. Contexto y límites

Archivos clave, tecnologías, reglas del negocio o qué cosas **no** deben modificarse.

3. Criterio de "Terminado"

Cómo se valida el éxito: tests pasando, script ejecutado sin errores o UI verificada en Chrome.

Prompt débil	Prompt de alto potencial
✗ "Arregla el script de precios porque falla."	✓ "Al ejecutar `actualizar_precios.py` falla al procesar Elektra. Reproduce el error, identifica la causa raíz sin romper los demás scrapers y verifica que genere el CSV correctamente."
✗ "Hazme una página web de reportes."	✓ "Usa `/plan` para diseñar un dashboard en HTML/Flask que lea los CSV de `data/`, muestre variación semanal de precios y luego pruébalo en el navegador."

EJEMPLO PRÁCTICO — PROMPT COMPLETO DE 3 PARTES

Así se ve una petición bien armada lista para ejecutar:

```
"Objetivo: Agregar una columna de variación porcentual semanal en el reporte HTML.
Contexto: Lee los datos desde historico.py y app.py sin cambiar el formato actual de los CSV en data/.
Terminado: El servidor local arranca sin errores y la tabla muestra la nueva columna con colores verde/rojo."
```

2 Comandos Slash (`/`) para cada escenario

Los comandos slash activan modos de trabajo especializados directamente desde el chat:

Comando	Cuándo usarlo y qué hace	Ejemplo real de uso
`/plan`	Tareas medianas/complejas. Diseña un plan paso a paso antes de editar archivos.	`/plan` Quiero migrar los CSV actuales a una base SQLite en `precios_db.py` sin perder compatibilidad con `app.py`.
`/grill-me`	Ideas abiertas. Te entrevista brevemente para cerrar decisiones de diseño.	`/grill-me` Quiero automatizar alertas cuando un competidor baje más de 10% su precio; ayúdame a definir reglas.
`/boost`	Problemas difíciles. Activa análisis profundo y verificación rigurosa.	`/boost` Optimiza `matching.py` porque tarda demasiado cruzando miles de productos y genera falsos positivos.
`/goal`	Tareas largas. Trabaja sin detenerse hasta cumplir todo el objetivo.	`/goal` Revisa todos los scrapers, estandariza el manejo de errores y agrega logs claros en cada uno.
`/teamwork`	Proyectos grandes divisibles en módulos con varios agentes en equipo.	`/teamwork` Refactoriza el backend en Flask, actualiza los templates HTML y genera la suite de pruebas en paralelo.
`/browser`	Pruebas web o interacción real con Chrome mediante DevTools.	`/browser` Abre `http://localhost:5000`, prueba los filtros de país y verifica que no haya errores en consola.
`/schedule`	Monitoreos o recordatorios periódicos en segundo plano.	`/schedule` Cada lunes a las 8:00 AM recuérdame revisar si algún scraper falló en `servidor.log`.
`/learn`	Guarda una corrección o regla para futuras conversaciones.	`/learn` Cada vez que generes reportes PDF en este proyecto, guárdalos dentro de la carpeta `reportes/`.
`/compact`	Chats muy largos. Resume el contexto para mantener precisión.	`/compact` Ya terminamos el módulo de precios; compacta para seguir con el módulo de exportación a PowerPoint.

3 Capacidades avanzadas para multiplicar tu trabajo

Código, archivos y terminal

Lectura selectiva de código, edición precisa multi-archivo y ejecución directa en PowerShell para correr scripts, pruebas, linters y validar salidas reales.

Subagentes especializados

research: explora repositorios extensos sin saturar el chat.
browser: controla Chrome para probar apps locales o sitios reales.
image-generator: crea o edita gráficos, íconos y mockups.

Swarms (Enjambres en paralelo)

Permite lanzar varios agentes trabajando simultáneamente en tareas independientes (o con revisión cruzada entre pares) sobre un tablero compartido.

Artefactos y documentos

Generación de reportes extensos, diagramas de arquitectura (Mermaid), planes interactivos y exportación a archivos locales como HTML, Markdown o PDF.

EJEMPLOS DE CAPACIDADES AVANZADAS

- **Subagente Browser:** *"Entra a la web del competidor en Chrome, revisa por qué no carga el precio en el HTML estático y dime qué selector CSS usa."*
- **Swarm en paralelo:** *"Divide en paralelo: un agente revisa los 4 archivos de scraping de El Salvador y otro los de Honduras para detectar selectores rotos."*
- **Skills de Datos / GCP:** *"Analiza este archivo CSV en un notebook* `.ipynb` *con gráficos de tendencia y resumen ejecutivo de bajas de precio."*
- **Plugins / Integraciones:** *"Conéctate con Google Sheets para exportar esta tabla comparativa directamente a una hoja de cálculo."*

4 Playbooks: cómo trabajar paso a paso

A. Crear una funcionalidad nueva

1. Usa `/grill-me` o `/plan` para definir alcance.
2. Pide implementación incremental por módulos.
3. Exige ejecución de pruebas o verificación en `localhost` con `/browser`.

B. Depurar un error difícil (Debugging)

1. Comparte el traceback completo y cómo reproducirlo.
2. Pide que primero reproduzca el fallo en terminal.
3. Aplica el fix y valida que no haya regresiones.

C. Análisis de datos y reportes

1. Indica la fuente (CSVs locales, BigQuery, GCS).
2. Define las preguntas de negocio clave.
3. Especifica el formato final: notebook, dashboard o PDF.

D. Automatizar tareas repetitivas

1. Describe la tarea manual (revisar 50 URLs, limpiar CSVs, renombrar archivos).
2. Pide un script reutilizable o delegación al agente de navegador.
3. Programa ejecuciones periódicas con `/schedule`.

EJEMPLO DE FLUJO COMPLETO (PLAYBOOK B — DEBUGGING)

```
"Al correr generar_resumen_bajas.py sale KeyError: 'precio_anterior'.
1) Ejecuta el script en terminal para reproducir el error.
2) Revisa los últimos CSV en data/ para ver si cambió el nombre de la columna.
3) Corrige el script manejando columnas faltantes y vuelve a correrlo para confirmar que
genera el resumen sin errores."
```

5 Antipatrones y Checklist rápido

Errores comunes que reducen la calidad de las respuestas

- **Mezclar proyectos distintos en un mismo chat:** abre una conversación nueva al cambiar de tema para mantener el contexto limpio.
- **Pedir cambios masivos sin plan previo:** divide en bloques verificables o inicia con `/plan`.
- **Dar por bueno el código sin ejecutarlo:** pide siempre que corra el script, los tests o revise la app en vivo.
- **Corregir lo mismo varias veces:** cuando ajustes una preferencia importante, usa `/learn` para fijarla.

EJEMPLO: CÓMO CORREGIR UN ANTIPATRÓN AL INSTANTE

- **En lugar de:** seguir en un chat de 80 mensajes pidiendo algo de otro proyecto → **Haz esto:** abre un chat nuevo o usa `/compact`.
- **En lugar de:** "*¿Seguro que funciona?*" → **Haz esto:** "*Ejecuta el script con datos reales y muéstrame la salida en consola.*"
- **En lugar de:** repetir cada semana "*pon los títulos en español*" → **Haz esto:** usa `/learn` para que quede guardado como regla.

Checklist antes de enviar tu próximo prompt

- ✓ ¿Dejé claro qué archivo, carpeta o sistema es el objetivo?
- ✓ ¿Indiqué las restricciones (qué no tocar o qué formato necesito)?
- ✓ ¿Definí cómo vamos a comprobar que quedó bien?
- ✓ ¿Elegí el comando adecuado (`/plan` , `/boost` , `/browser`) si la tarea lo amerita?